Share Your Experiences
Python Fundamentals
Python Data Structures
Advanced Python
Data Science with Python
Web Development with Python
Python Practice
Data Science Course Explore

# Building a Simple API with Django REST Framework

Last Updated : 7 Apr, 2026

Django REST Framework (DRF) is an extension of Django that makes building APIs easier. It helps convert Django models into RESTful APIs that can be used by web apps, mobile apps, or other services.

Before creating an API, it's important to understand three main components:

- **Serialization:** Converts complex data, such as Django models or querysets, into JSON or XML format.
- **Viewsets:** Define views that handle API requests and responses.
- **URL Routing:** Maps URLs to API views so clients can access them.

## A simple CRUD API

Consider a Django project with an app that includes a Book model containing three fields: title, author and publish_date.

### Step 1: Add DRF to Installed Apps

In settings.py, add "rest_framework" and the app name to "INSTALLED_APPS" to enable Django REST Framework in the project:

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'app'
]
```

Make sure the Django REST Framework module is installed:

```
pip install djangorestframework
```

### Step 2: Create the Book Model

In **app/models.py**, define the Book model with three fields:

```
from django.db import models

class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.CharField(max_length=100)
    publish_date = models.DateField()

    def __str__(self):
        return self.title
```

- **CharField** is used for short text (title, author).
- **DateField** stores the date when the book was published.
- **__str__** method returns a human-readable representation of each book instance.

### Step 3: Create a Serializer

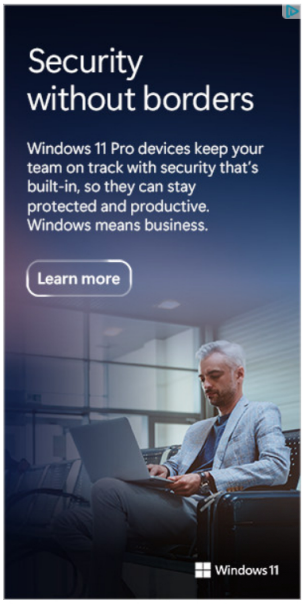
Serializers convert model instances into JSON and validate incoming JSON data.

Create **app/serializers.py**:

```
from rest_framework import serializers
from .models import Book

class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = ['id', 'title', 'author', 'publish_date']
```

- **ModelSerializer** automatically generates fields based on the model.
- **fields** tuple specifies which model fields to expose via the API.




Upcoming Courses
MERN Full Stack...
Starting from - April 25, 2026
LIVE
4.7
System Design Traini...
Starting from - April 26, 2026
LIVE
4.6
Tech Interview 101 -...
Starting from - May 2, 2026
LIVE
4.9
View All →

Fresher Jobs Experienced Jobs
Trainee Software Engineer
Nirvana Solutions
Internship • Hybrid - Gurgaon (Haryana)
Apply by Sat Apr 25 2026
AI Intern
Onsurity
Internship • Onsite - Bengaluru...
Apply by Sat Apr 25 2026
Junior Project Coordinator
LBM Solutions
0.5 - 1 Years • Onsite - Mohali (Punjab)
Apply by Thu Apr 30 2026
View All →

- **id** field (primary key) is included to uniquely identify each record.

### Step 4: Define a ViewSet

ViewSets provide CRUD operations in one place.

In **app/views.py**:

```
from rest_framework import viewsets
from .models import Book
from .serializers import BookSerializer

class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

- **ModelViewSet** automatically provides list, retrieve, create, update and destroy actions.
- **queryset** defines the set of objects available via the API.
- **serializer_class** tells DRF how to serialize/deserialize the Book data.

> ***Note:*** *You do not need to manually convert data to JSON. The BookSerializer handles serialization and deserialization automatically when using ModelViewSet.*

### Step 5: Create app URLs with a DRF router

DRF routers automatically generate URL patterns for your ViewSets.

Create **app/urls.py**:

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import BookViewSet

router = DefaultRouter()
router.register(r'books', BookViewSet)

urlpatterns = [
    path('', include(router.urls)),
]
```

Using DefaultRouter, don't need to manually define each URL for CRUD operations. It automatically creates standard RESTful endpoints like:

- **GET** /books/ (list all books)
- **POST** /books/ (create new book)
- **GET** /books/{id}/ (retrieve a single book)
- **PUT** /books/{id}/ (update a book)
- **DELETE** /books/{id}/ (delete a book)

Including these URLs in the project's main urls.py file exposes the API endpoints.

### Step 6: Migrate and Run the Server

Run the following commands to create your database tables and start the server:

```
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

### Step 7: Include app URLs from the project URLs

In **project/urls.py:**

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('app.urls')),
]
```

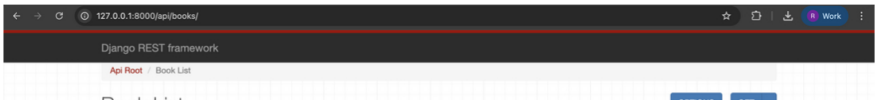
Once the server is running:

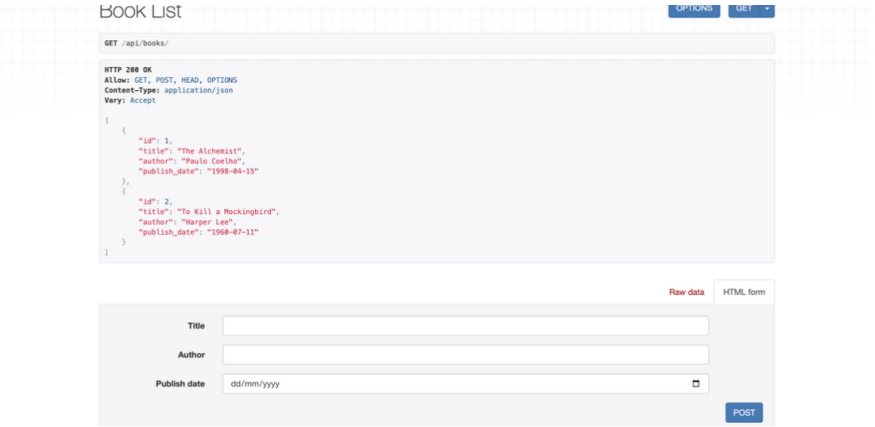
- The **/books/** endpoint lists all books (initially empty).
- Use **POST** requests to add new books.
- Individual books can be retrieved, updated, or deleted via their respective URLs.

### Accessing the API Endpoints

Including the app URLs under **path('api/', include('app.urls'))** in the main urls.py adds the **/api/** prefix to every route defined inside the app. As a result, all API endpoints automatically start with **/api/.**

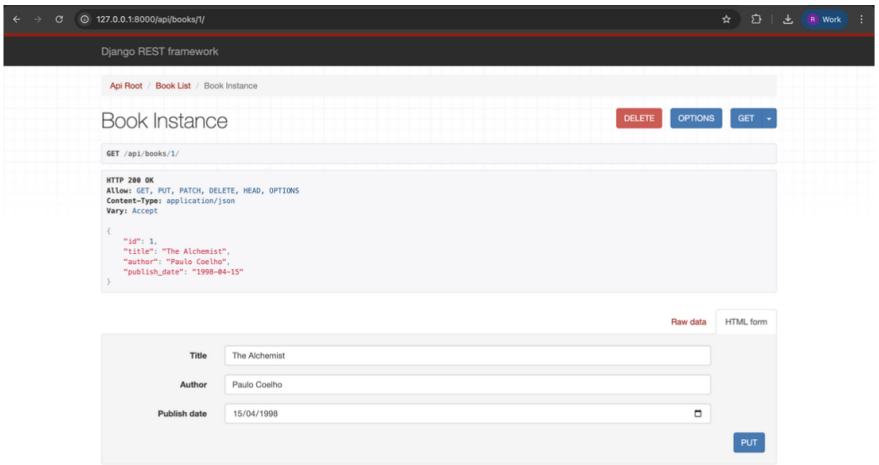
**Output: /books/** endpoint after adding some entries.






*Snapshot after adding two entries*

**Output: /books/{id}/** with id of some book.




*Snapshot of books id= 1.*

You can perform all CRUD operations using the following endpoints:

| HTTP Method | Endpoint URL | Description |
|---|---|---|
| GET | http://127.0.0.1:8000/api/books/ | List all books |
| POST | http://127.0.0.1:8000/api/books/ | Create a new book |
| GET | http://127.0.0.1:8000/api/books/{id}/ | Retrieve a book by its ID |
| PUT | http://127.0.0.1:8000/api/books/{id}/ | Update a book by its ID |
| DELETE | http://127.0.0.1:8000/api/books/{id}/ | Delete a book by its ID |

These endpoints show that all CRUD operations are available and functioning correctly in the application.

**Suggested Quiz** 5 Questions

Which three main components are required before creating an API in DRF?

(A) Serialization, Viewsets, URL Routing

(B) Models, Templates, Forms

(C) Admin, Middleware, Signals

(D) Static Files, Templates, URLs

Login to View Explanation 1/5 < Previous Next >

Comment

Navee...

29

**Article Tags:** Python | Python Django | Django-REST

GeeksforGeeks
Sanchhaya Education Private Limited

**Corporate & Communications Address:**
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

**Registered Address:**
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

**Company**
About Us
Legal
Privacy Policy
Careers
Contact Us
Corporate Solution
Campus Training Program

**Explore**
POTD
Practice Problems
Connect
Blogs
Upskill Courses

**Tutorials**
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
GATE
School Subjects
Software and Tools

**Courses**
ML and Data Science
DSA and Placements
Web Development
Data Science
Programming Languages
DevOps & Cloud
GATE
MongoDB Certifications

**Offline Centers**
Noida
Bengaluru
Hyderabad
Kolkata

**Preparation Corner**
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

GET IT ON Google Play
Download on the App Store

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved